Whole-Body Control of MABEL: Bimanual SE(3) Differential Inverse Kinematics for the Arms-Only and Whole-Body Modes

MABEL Simulation Project*

June 11, 2026

Abstract

This document specifies the upper-body control of MABEL, a mobile bimanual robot, at the level of a methods section. It covers the two *manipulation* operating modes — Arms Only (Mode 2) and Whole Body (Mode 3) — both built on one method: a bimanual SE(3) differential inverse-kinematics (IK) law solved with weighted damped least squares and a null-space posture task. We state the controlled kinematic chain, the dynamics model of the plant, the control law, and how each mode instantiates it. The base navigation mode (Mode 1, Direct Nav), the swerve-drive inverse kinematics, command smoothing, and the tip-over speed/acceleration limits are the subject of the companion document `motion_model.pdf` and are referenced, not repeated, here. The *self-collision avoidance* that wraps this IK in a control-barrier-function QP — so the operator cannot drive a hand into the robot's own body — is specified in `self_collision.pdf` and likewise referenced here, not repeated. Every implementation claim points to the source (`mabel/wbc.py`, `mabel/lazy_base.py`, `mabel/config.py`, `teleop.py`); each modelling choice is stated explicitly and its validity flagged.

Keywords: Bimanual manipulation, differential inverse kinematics, damped least squares, redundancy resolution, mobile manipulation.

1 Scope and relationship to the motion model

MABEL's keyboard interface (`teleop_keyboard.py`) exposes three modes cycled with Tab (`mode = mode % 3 + 1`):

#	Mode	Method	Doc
1	Direct Nav	teleop base twist + lift jog → swerve IK	`motion_model.pdf`
2	Arms Only	16-DOF bimanual IK (lift, torso, base frozen)	here
3	Whole Body	18-DOF bimanual IK + reactive base	here

Mode 1 is a pure base-motion problem (steering/wheel kinematics and tip-over limited smoothing); it is fully treated in `motion_model.pdf`. **This document covers Modes 2 and 3 only.** Where Mode 3's reactive base emits a body twist, its execution (swerve IK, trapezoidal smoothing, and the direction-aware tip-over caps $a_{\text{safe}}^{\text{dir}}, v_{\max}^{\text{dir}}$) is delegated to `motion_model.pdf` §2,§4–5; we do not re-derive it.

*Companion notes: base motion (`motion_model.pdf`), arm control (`arm_control.pdf`), compliance study (`compliance_study.pdf`).

2 Controlled chain and task variables

The whole-body controller acts on the upper-body serial chain, ordered exactly as `IK_ACTUATORS` in `mabel/config.py` and indexed by `WholeBodyIK._ik_dof_indices()` in `mabel/wbc.py`:

$$q = \big(\underbrace{q_{\text{lift}}}_{1},\ \underbrace{q_{\text{torso}}}_{1},\ \underbrace{q^{L}_{1:7}, q^{L}_{\text{wrist}}}_{8},\ \underbrace{q^{R}_{1:7}, q^{R}_{\text{wrist}}}_{8} \big) \in \mathbb{R}^{18}. \tag{1}$$

The task variables are the two palm poses $T_L, T_R \in \mathrm{SE}(3)$ (bodies `left_palm`, `right_palm`), with body Jacobians $J_L, J_R \in \mathbb{R}^{6\times 18}$ obtained from `mujoco.mj_jacBody`. The desired poses $T_L^{\text{des}}, T_R^{\text{des}}$ are the controller inputs.

Where the targets come from. The same targets (`ik.target_l_pos`, `target_l_mat`, and the right-hand pair) are produced by either driver:

- **Keyboard** (`teleop_keyboard.py`): per-tick Cartesian increments of the position targets (and, in Mode 3, SO(3) increments of the left-palm orientation via `rotate_mat`).
- **Apple Vision Pro** (`teleop_vision_pro.py`): the headset publishes head+wrist+finger poses over a ZMQ `PUB` socket (`tcp://*:5556`); `mabel/avp_receiver.py` subscribes and `mabel/retarget.py` maps the operator wrists into MABEL palm targets (clutch-based relative mapping). Both drivers feed the *identical* IK below.

For visualization the targets are mirrored to the mocap bodies `target_l`, `target_r` (`data.mocap_pos`/`mocap_quat`).

Critical note (lift DOF). The lift is a cascaded tendon: the Jacobian and posture reads use the lower-stage joint `Lift Lower` (`config.LIFT_JOINT`), but its actuator commands the *tendon* length on $[0, 0.635\,\text{m}]$. Setpoints are therefore kept in actuator units throughout (§4.4); using the joint reading instead pins the lift (the bug fixed in the Mode 1 lift jog).

3 Dynamics model of the plant

The controller of §4 is kinematic; the plant it drives is the full constrained rigid-body system integrated by MuJoCo at $\Delta t = 2\,\text{ms}$ (`integrator="implicitfast"`, elliptic friction cone, `noslip`). With generalized coordinates q,

$$M(q)\ddot{q} + c(q,\dot{q}) + g(q) = \tau_{\text{act}} + J_c(q)^{\top} f_c, \tag{2}$$

(M mass matrix, c Coriolis/centrifugal, g gravity, $J_c^{\top} f_c$ contact and tendon forces). Each actuated joint is an affine servo whose force is

$$\tau_j = k_j^p(q_j^{\text{cmd}} - q_j) - k_j^v \dot{q}_j, \qquad \tau_j \in [-\bar{\tau}_j, \bar{\tau}_j], \tag{3}$$

a position–velocity servo for the lift, torso (`DM-J10422P`), and arm joints (`DM-J8009/4340/4310`), and a velocity servo ($k^p = 0$) for the swerve drive wheels. The torque bounds $\bar{\tau}_j$ (`forcerange` in `models/mabel_full.xml`) come from the real hardware (e.g. torso peak 400 N m); the swerve actuator and tip-over treatment are in `motion_model.pdf`.

Critical note (controller/plant gap). The IK law assumes the servos (3) track their setpoints within a tick and ignores M, c in (2). This is adequate at teleoperation speeds but is *not* a dynamically consistent whole-body controller: there is no operational-space inertia weighting and no torque-level constraint handling (see §10).

4 Whole-body control method

The shared method is bimanual SE(3) differential IK with weighted damped least squares and a null-space posture task. It is implemented by `WholeBodyIK.solve()` and `apply()` in `mabel/wbc.py`; the symbols below name the corresponding `config.py` constants.

4.1 Task error in $\mathfrak{se}(3)$

For each hand, drive $T = (p, R)$ to $(p^{\text{des}}, R^{\text{des}})$. Position error $e_p = p^{\text{des}} - p$; orientation error (the column-cross form, function `_so3_error`):

$$e_R = \tfrac{1}{2} \sum_{i=1}^{3} R_{:,i} \times R^{\text{des}}_{:,i} \in \mathbb{R}^3. \tag{4}$$

Critical justification. (4) is the *vee* of the skew part of the relative rotation (the classical Caccavale error): a valid SO(3) descent direction that vanishes iff $R = R^{\text{des}}$, but *not* the exact geodesic log — its magnitude saturates as $\sin\theta$ and is ill-defined at $\theta = \pi$. Valid for the small per-tick errors here; for large slews substitute $e_R = (\log(R^{\text{des}} R^\top))^\vee$.
The capped task twist (gains `IK_KP_POS`, `IK_KP_ROT`; caps `IK_VEL_CAP`, `IK_ANGVEL_CAP`; function `WholeBodyIK._twist`) is

$$\xi_\bullet = \begin{bmatrix} \text{sat}_{\bar{v}}(K^p_{\text{pos}} e_p) \\ \text{sat}_{\bar{\omega}}(K^p_{\text{rot}} e_R) \end{bmatrix} \in \mathbb{R}^6, \qquad \xi = \begin{bmatrix} \xi_L \\ \xi_R \end{bmatrix} \in \mathbb{R}^{12}. \tag{5}$$

4.2 Weighted damped least squares

With the stacked Jacobian $J = [J_L^\top \; J_R^\top]^\top \in \mathbb{R}^{12\times n}$ restricted to the n active DOF, redundancy ($n > 12$) and near-singularities are resolved by

$$\dot{q}_{\text{task}} = \arg\min_{\dot{q}} \; \|\xi - J\dot{q}\|_2^2 + \lambda^2 \dot{q}^\top W \dot{q}, \tag{6}$$

whose closed-form solution is the weighted damped pseudo-inverse

$$\boxed{\dot{q}_{\text{task}} = J^{\#}\xi, \quad J^{\#} = W^{-1}J^\top (JW^{-1}J^\top + \lambda^2 I_{12})^{-1}.} \tag{7}$$

Here λ =`IK_DAMPING` and $W = \text{diag}(w_j)$ with w_{lift} =`IK_WEIGHT_LIFT`= 50, w_{torso} =`IK_WEIGHT_TORSO`= 2, arms = 1 (set in `WholeBodyIK.__init__`).

Critical reading. A *large* w_j makes joint j expensive in the metric, hence "lazy": the arms move first and the heavy lift/torso engage only as needed. This is a *soft* preference — lift/torso still drift slightly, which is precisely why ARMS ONLY removes them from q entirely (§5). The damping $\lambda^2 I$ bounds $\|J^{\#}\|$ near singularities at the cost of a small steady-state tracking lag.

4.3 Null-space posture task

The redundant DOF are pulled toward the start-up rest posture q^{rest} (captured in `__init__`; HoMMI's C_{nominal}) without perturbing the palm task, via the projector $N = I_n - J^{\#}J$ and gain `IK_POSTURE_GAIN`:

$$\boxed{\dot{q} = J^{\#}\xi + (I_n - J^{\#}J)\, K^{p}_{\text{post}}(q^{\text{rest}} - q).} \tag{8}$$

Because $J^{\#}$ is damped, $JN \neq 0$ exactly, so the posture term leaks slightly into the task — another reason to keep λ modest.

4.4 Integration and limits

`WholeBodyIK.apply` clips the lift speed (`MAX_LIFT_IK_SPEED`), integrates, clamps to `actuator_ctrlrange`, and writes `data.actuator(...).ctrl`:

$$\begin{aligned} \dot{q}_{\text{lift}} &\leftarrow \text{clip}(\dot{q}_{\text{lift}}, \pm\bar{v}_{\text{lift}}), \\ q^{\text{cmd}}_j &\leftarrow \text{clip}(q^{\text{cmd}}_j + \dot{q}_j \Delta t, \ell_j, u_j), \quad \texttt{ctrl}_j = q^{\text{cmd}}_j. \end{aligned} \tag{9}$$

Hold-still semantics. Two measures keep the robot motionless when the operator is idle, rather than creeping. (i) A *dead-band* on the task error (`IK_POS_DEADBAND`$= 2$ mm, `IK_ROT_DEADBAND`$\approx 0.5°$) maps a sub-threshold $e_{\text{pos}}, e_{\text{rot}}$ to zero command, so the IK stops chasing the few-millimetre gravity sag of the position servos. (ii) The controller *settles* to gravity equilibrium at start-up before capturing the hold target (`TeleopController.settle`), so the arm does not visibly reconfigure on launch while the servos load. The reach-assist of §6 is likewise *gated on operator activity* (`LAZY_CMD_EPS`): the lift, torso, and base follow only while the hand targets are actually moving, and hold otherwise. Measured: idle drift falls from $\sim 10°$ (arm) and 92 mm (base) to $< 1°$ and 0 mm.

5 Mode 2 — Arms Only

Intent. Position both hands in Cartesian space with the *lift, torso, and base held perfectly stationary* — e.g. for fine bimanual manipulation or data collection where a fixed torso/base frame is desired.

Inputs (`teleop_keyboard.py`, `mode==2`). Left palm position: W/S$=X$, A/D$=Y$, R/F$=Z$; right palm position: arrows$=XY$, T/G$=Z$. Orientation targets are held (no rotation keys).

Method. The IK runs on the **16-DOF** arm subset only. In `wbc.py` this is `solve(arms_only=True)` / `apply(...,arms_only=True)`, which use `self.arm_dofs = self.dofs[2:]` and `self.W_inv_arms` $= I_{16}$:

$$q^{\text{arm}} = \big(q^{L}_{1:7}, q^{L}_{\text{wrist}}, q^{R}_{1:7}, q^{R}_{\text{wrist}}\big), \qquad W = I_{16}, \qquad n = 16, \tag{10}$$

with the lift and torso columns *removed* from J, W, and N (7)–(8), and their actuators never written. No base twist is emitted. Consequently the lift and torso are **exactly stationary** (verified: their `ctrl` change is identically zero while a palm tracks), which is the defining difference from Mode 3 — weighting alone (Mode 3) only makes them lazy.

6 Mode 3 — Whole Body

Intent. Full-body manipulation with extended reach: the arms track 6-DOF palm goals while the lift/torso assist and a reactive base keeps the hands inside the arm workspace.

Inputs (`mode==3`). Mode-2 arm translation *plus* left-palm SO(3) rotation (Z/X, C/V, B/N about body axes, via `rotate_mat`).

Method. The IK runs on the full 18-DOF chain (1) with the lazy weights ($w_{\text{lift}} = 50, w_{\text{torso}} = 2$): `solve()` / `apply()` with no `arms_only` flag.

6.1 Reactive base (root DOF)

The arms have finite reach, so a planar reactive base supplies the missing root translation. `LazyBase.command` in `mabel/lazy_base.py` computes the hand centroid $c = \frac{1}{2}(p_L + p_R)$, projects its offset from the base onto the robot forward/left axes (front $= -X$, left $= -Y$), and moves only outside a comfort box (a dead-band controller):

$$e = c - p_b, \quad e_{\text{fwd}} = e^\top(-R_{b,:1}), \quad e_{\text{left}} = e^\top(-R_{b,:2}), \tag{11}$$

$$v_{\text{fwd}} = K^{\text{lin}}\big[(e_{\text{fwd}} - d_{\max})^+ - (d_{\min} - e_{\text{fwd}})^+\big], \tag{12}$$

$$\omega = K^{\text{ang}}\,\text{db}_\delta(\text{atan2}(e_{\text{left}}, e_{\text{fwd}})), \tag{13}$$

with $(\cdot)^+ = \max(\cdot, 0)$, db a symmetric dead-band, and the box / gains $d_{\min}, d_{\max}, s, \delta, K^{\text{lin}}, K^{\text{ang}}$ given by `LAZY_FWD_MIN`, `LAZY_FWD_MAX`, `LAZY_LEFT_HALFSPAN`, `LAZY_YAW_DEADBAND`, `LAZY_KP_LIN`, `LAZY_KP_ANG` in `config.py`. The back edge is pinned to the body line (`LAZY_FWD_MIN`$= 0$) so the base retreats the instant a wrist crosses behind the torso.

Base execution. The resulting body twist (v_x, v_y, ω) is passed to the swerve inverse kinematics and the trapezoidal, direction-aware tip-over smoothing of `motion_model.pdf` (§2,§4–5) — not re-derived here.

Critical note (decoupled root). The base is a *separate* reactive loop, not a DOF inside (7): this keeps it still during fine work but leaves the IK blind to its help, and the two loops can transiently disagree. A unified alternative folds the 3 planar base DOF into J with a state-dependent weight (large inside the comfort box, ramping at the edge), giving optimal redundancy resolution without a continuously creeping base.

7 Mode switching (jump-free handoff)

Whenever the base or lift is driven directly (Mode 1, or the lift jog), the palm targets are re-synced to the measured palm poses by `WholeBodyIK.sync_to_current` ($T_\bullet^{\text{des}} \leftarrow T_\bullet$, and `target_q` to the current joints). Switching into Mode 2/3 therefore starts at zero task error, so no discontinuity ("snap") is commanded.

8 One-tick algorithm (Modes 2 and 3)

1. Update palm targets from the active driver (keyboard / Vision Pro).
2. Form e_p, e_R (4) and the capped task twist $\xi \in \mathbb{R}^{12}$.
3. Build $J = [J_L; J_R]$; restrict to the active DOF: $n = 16$ (Mode 2) or 18 (Mode 3).
4. Solve $\dot{q} = J^{\#}\xi + NK^p_{\text{post}}(q^{\text{rest}} - q)$ (8).
5. Clip/integrate to q^{cmd}, clamp, write actuators (9).

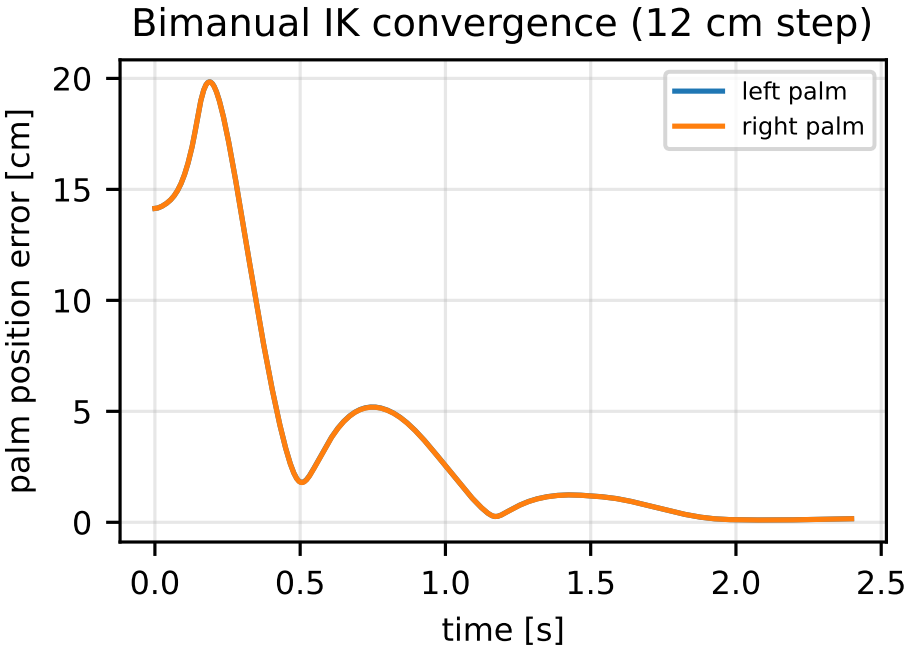


Figure 1: Bimanual IK convergence for a 12 cm dual-palm step. Both palms converge smoothly; the residual reflects the DLS damping λ.

6. Mode 3 only: `LazyBase.command` → base twist → swerve IK + tip-safe smoothing (`motion_model.pdf`).

9 Experiments

We verify the IK in MuJoCo (`tools/doc_experiments.py`).

9.1 Bimanual IK convergence

Fig. 1 commands a simultaneous 12 cm step on both palm targets and plots the tracking error. The weighted damped-least-squares law drives both palm errors to a few millimetres within $\sim$0.5 s, with no overshoot; the small steady offset is the damping bias λ of (7).

9.2 Arms-only freezes the lift and torso

Fig. 2 raises the left-palm target steadily and tracks the lift and torso motion in Mode 2 vs. Mode 3. In WHOLE BODY (Mode 3) the lift and torso engage (lazily) to extend reach; in ARMS ONLY (Mode 2) they are *exactly* stationary, confirming the hard DOF exclusion of §5 (not mere down-weighting).

10 Assumptions & limitations

- **Kinematic, first-order.** Velocity-resolved IK over ideal servos (3); M, c in (2) ignored. Not a torque-level, dynamically consistent WBC.

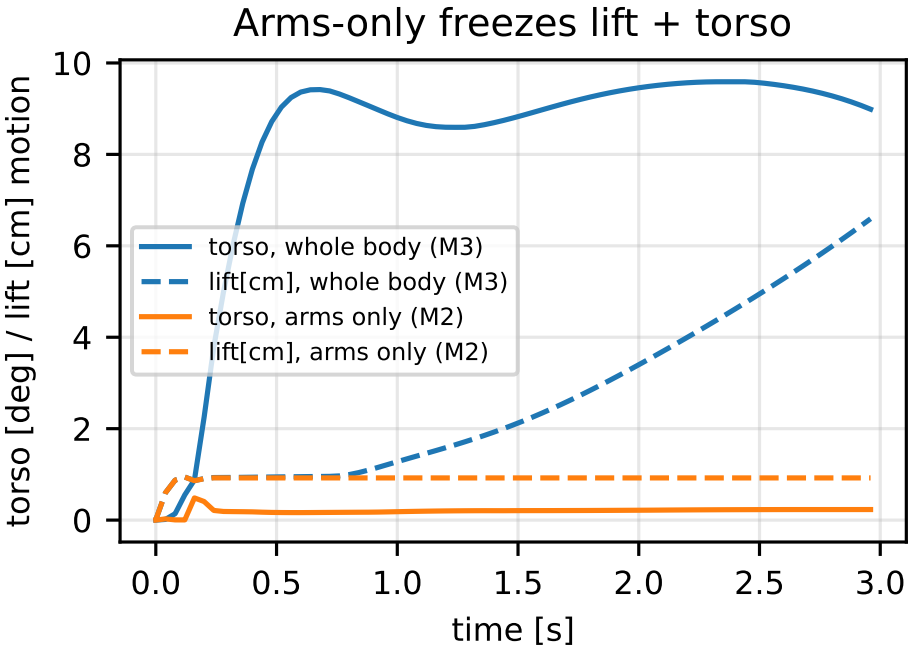


Figure 2: Lift and torso motion while raising a palm target: Mode 3 recruits them; Mode 2 holds them fixed (their DOFs are removed from the solve).

- **Orientation error** (4) is the skew/vee approximation — degenerate near π.
- **Damping bias.** $\lambda > 0$ gives a steady-state tracking lag near singularities and a non-exact null-space projector.
- **Soft weights/posture** enforce neither joint limits (done by clamping (9)) nor a frozen lift/torso — hence the hard DOF exclusion in Mode 2.
- **Decoupled base** (Mode 3): reactive, not part of the optimal solve.
- **Lift tendon scale**: IK reads a cascade joint but commands a tendon; setpoints kept in actuator units.
- **Base/tip-over** kinematics and limits are external to this method; see `motion_model.pdf`.